

Eigenes Projekt: BillSplitter

Einleitung:

Diese Dokumentation beschreibt die von mir eigenständig entwickelte BillSplitter-App. Die Anwendung ermöglicht es, einen Rechnungsbetrag inklusive Trinkgeld wahlweise gleichmäßig oder individuell auf mehrere Personen aufzuteilen. Alle Eingaben (Rechnungsbetrag, Trinkgeldprozentsatz, Personenzahl und individuelle Anteile) werden persistent in Android SharedPreferences gespeichert und bei Bedarf automatisch proportional neu verteilt. Die App wurde vollständig allein in Kotlin mit Jetpack Compose realisiert, ohne Mitwirkung weiterer Teammitglieder.

1. User Storys

Must	Als Nutzer möchte ich eine Rechnungssumme eingeben, um sie splitten zu können
Must	Als Nutzer möchte ich den Trinkgeld-Prozentsatz einstellen (z.B. per Slider oder Dropdown)
Must	Als Nutzer möchte ich die Anzahl der Personen einstellen, um den Pro-Kopf-Betrag ermitteln zu können
Should	Als Nutzer möchte ich, dass die App beim Neustart letzte Werte wieder herstellt (SharedPreferences)
Should	Als Nutzer möchte ich per „Reset“-Button alle Eingaben zurücksetzen können
Could	Als Nutzer möchte ich vordefinierte Trinkgeld-Buttons (10%, 15%, 20% ...) schnell wählen können
Could	Als Nutzer möchte ich die Namen der beteiligten Angeben können
Could	Als Nutzer möchte einstellen können, wie die Rechnung geteilt wird (z.B. jeder Zahlt den gleichen Anteil, oder eine Person zahlt einen gewissen Betrag der Rest wird unter den anderen aufgeteilt etc.)

2. Benutzeroberfläche (UI-Elemente):

5:20 5G

Rechnung (€)

Trinkgeld: 0 %

5 % 10 % 15 % 20 %

Personen: - 1 +

Split-Modus: Gleichmäßig ☒ Individuell

Trinkgeld: € 0.00

Gesamt: € 0.00

Pro Person: € 0.00

Reset

1. Startbildschirm

5:23 5G

Rechnung (€) 20

Trinkgeld: 10 %

5 % 10 % 15 % 20 %

Personen: - 3 +

Split-Modus: Gleichmäßig ☒ Individuell

Trinkgeld: € 2.00

Gesamt: € 22.00

Pro Person: € 7.33

Reset

2. Rechnung gleichmäßig Verteilt

5:27 123 5G

Rechnung (€)

20

Trinkgeld: 10 %

5 % 10 % 15 % 20 %

Personen: - 4 +

Split-Modus: Gleichmäßig ☒ Individuell

Name	%	€
Diego	10	2.20
Michael	20	4.40
Andy Rubin	35	7.70
Nick Sears	35	7.70

Offen: 0% (-€0.00)

Trinkgeld: € 2.00

Gesamt: € 22.00

Reset

3. Korrekte Berechnung mit individuellem Split

5:30 5G

Rechnung (€)

20

Trinkgeld: 10 %

5 % 10 % 15 % 20 %

Personen: - 4 +

Split-Modus: Gleichmäßig ☒ Individuell

Name	%	€
Diego	10	2.20
Michael	20	4.40
Andy Rubin	80	17.60
Nick Sears	30	0.00

Zu viel: 10% (€2.20)

Trinkgeld: € 2.00

Gesamt: € 22.00

Reset

4. Individuelle Berechnung mit Warnung

3. Benutzeroberfläche (UI-Elemente):

- Rechnungsbetrag
 - Texteingabefeld für den Gesamtbetrag
- Trinkgeld
 - Prozent-Eingabe (Nummernfeld, Default = 0%)
 - Slider
 - Vordefinierte Buttons: 5%, 10%, 15%, 20%
- Personenzahl:
 - Anpassung über „-“ / „+“ Buttons
- Split-Modus (Schiebeschalter, Default = Gleichmäßig)
 - Gleichmäßig
 - Anzeige des gesamten Trinkgeldbetrags
 - Anzeige des Gesamtbetrags inkl. Trinkgeld
 - Anzeige des Betrags pro Person inkl. Trinkgeld
 - Reset-Button
 - Individuell
 - Eine Zeile pro Person mit je:
 - Name
 - Anteil in %
 - Anteil in Euro
 - Scrollbar Personenliste (Bei hoher Personenzahl)
 - Stets sichtbare Gesamtübersicht unter der Liste:
 - Offener/Überschüssiger (in % und Euro)
 - Gesamtes Trinkgeld (in Euro)
 - Gesamtbetrag inkl. Trinkgeld (in Euro)
 - Reset-Button

4. Bedienungsablauf

1. App starten
 - Der Anwender sieht den Startbildschirm mit den oben aufgeführten UI-Elementen
2. Rechnungsbetrag und Trinkgeld festlegen
 - Eingabe des Rechnungsbetrags
 - Auswahl des Trinkgelds per Zahlenfeld, Schieberegler oder vordefinierte Buttons
3. Personenzahl einstellen
 - „-“ und „+“ Buttons erhöhen oder verringern die Anzahl der Personen
4. Split-Modus auswählen
 - Gleichmäßig (Standard)
 - Direkte Anteile von
 - Trinkgeld in Euro
 - Gesamtbetrag inkl. Trinkgeld
 - Betrag pro Person inkl. Trinkgeld in Euro
 - Individuell:
 - Für jede Person wird eine Zeile angezeigt
 - Eingabe von Name, Prozent-Anteil und Euro-Betrag.

- Bei mehr Personen als Platz vorhanden, bleibt die Gesamtübersicht (offener Betrag, Trinkgeld gesamt, Gesamtbetrag inkl. Trinkgeld) stets sichtbar
 - Automatische Anpassung:
 - Wird der Betrag einer Person manuell geändert, passen sich alle anderen Beträge automatisch proportional an.
 - Festgelegte Werte bleiben danach fixiert, auch bei weiteren Änderungen
 - Damit wird das sogenannte Constraint-Appportionment-Problem gelöst
 - Über- oder Unterdeckung:
 - Wenn die Summe der individuellen Anteile nicht genau 100% ergibt, zeigt ein rotes Textfeld den offenen bzw. überschüssigen Betrag in % und Euro an.
5. Endgültige Übersicht
- Das Gesamtergebnis (gesamtes Trinkgeld und Gesamtbetrag inkl. Trinkgeld) wird in beiden Modi identisch dargestellt
6. Reset
- Über den Reset-Button am untern Bildschirmrand werden alle Einstellungen auf die Default-Werte zurückgesetzt und zuvor fixierte Beträge wieder zur automatischen Berechnung freigeben

5. Datenpersistenz:

- Dauerhafte Speicherung aller Eingaben (Teilnehmerliste) in Android SharedPreferences unter „billsplit_prefs“ als JSON-String.
 1. Initiales Laden
 - Zugriff auf SharedPreferences("billsplit_prefs", MODE_PRIVATE)
 - Abruf mit getString("participants", "[]") und Parson in List<Participant> (Name + Gewicht) in einem mutableStateOf
 2. Automatisches Speichern
 - Beobachtung von Listenänderungen mittels LaunchedEffect(participants).
 - Serialisierung mit JSONArray(participants.map { it.toJson() }).toString() und editor.apply().
 3. (De-)Serialisierung
 - toJson(participant) erzeugt JSONObject mit name und weight.
 - parseParticipantsFromJson(jsonString) wandelt JSONArray in Participant-Objekte um.
- Scope
 - Persistenz bei App-Kills und reboots
 - UI-State wie Rechnungsbetrag und Trinkgeld-Eingabe bleiben mit rememberSaveable über Konfigurationsänderungen erhalten.

6. Klassendokumentation

1. BillSplitCalculator

Paket: com.example.billsplitter.model

Typ: data class

Konstruktorparameter:

Name	Typ	Beschreibung
billAmount	Double	Rechnungsbetrag (in Euro)
tipPercent	Int	Trinkgeld-Prozentsatz (in %)
partySize	Int	Anzahl der Personen, auf die geteilt wird

Methoden:

- *fun tipTotal(): Double*
Berechnet das Gesamttrinkgeld in Euro
- *fun totalWithTip(): Double*
Berechnet den Gesamtbetrag inkl. Trinkgeld
- *fun perPerson(): Double*
Berechnet den Betrag pro Person

2. Participant

Paket: com.example.billsplitter.model

Typ: data class

Konstruktorparameter

Name	Typ	Beschreibung
Name	String	Name des Teilnehmers
percentShare	Int	Prozentualer Anteil am Gesamtbetrag
isFixed	Boolean	Gibt an, ob der Anteil fest (nicht mehr anpassbar) ist (Default = false)

Methoden:

- *fun amount(totalWithTip: Double): Double*
Berechnet den Eurobetrag eines Teilnehmers
- *fun withAmount(totalWithTip: Double, amount: Double): Participant*
Erzeugt eine Kopie mit neu berechnetem percentShare (gerundet, begrenzt auf 0-100) und setzt isFixed=true

3. BillSplitApp

Paket: com.example.billsplitter.ui

Typ: @Composable-Funktion (Root der UI)

Signatur:

@Composable

fun BillSplitApp()

Zustandsvariablen:

- val context: Context (über LocalContext.current)
- val prefs: SharedPreferences (gespeichert unter "bill_split_prefs")
- var billText: String (MutableState, initial aus KEY_BILL)
- var tipPercent: Int (MutableState, initial aus KEY_TIP)
- var peopleCount: Int (MutableState, initial aus KEY_PEOPLE)
- val participants: SnapshotStateList<Participant> (MutableStateList, initial aus KEY_PARTICIPANTS JSON)
- var splitMode: SplitMode (MutableState, initial aus KEY_SPLIT_MODE)

Persistenz:

- Laden:
 - o prefs.getString("KEY_BILL", "") → billText
 - o prefs.getInt("KEY_TIP", 15) → tipPercent
 - o prefs.getInt("KEY_PEOPLE", 1) → peopleCount
 - o prefs.getString("KEY_PARTICIPANTS", null) → parse JSON → participants
 - o prefs.getString("KEY_SPLIT_MODE", SplitMode.EQUAL.name) → splitMode
- Speichern:
 - o In LaunchedEffect-Blöcken für billText, tipPercent, peopleCount, participants, splitMode wird jeweils prefs.edit().put...().apply() aufgerufen.

4. ParticipationRow

Paket: com.example.billsplitter.ui

Typ: @Composable-Funktion für einzelne Zeile im Individual-Modus

Signatur:

@Composable

```
fun ParticipationRow(  
    participant: Participant,  
    totalWithTip: Double,  
    sumOthersPercent: Int,  
    onChange: (Participant) -> Unit  
)
```

Parameter:

- participant: Participant – aktueller Teilnehmer
- totalWithTip: Double – Gesamtbetrag inkl. Trinkgeld
- sumOthersPercent: Int – Summe der Anteile aller anderen Teilnehmer

- onChange: (Participant) -> Unit – Callback bei Anpassung

Lokaler Zustand:

- percentText: String (MutableState, Anzeige %-Feld)
- percentFocused: Boolean (MutableState)
- amountText: String (MutableState, Anzeige €-Feld)
- amountFocused: Boolean (MutableState)

Verhalten:

1. Zwei OutlinedTextFields für % und €:
 - Eingabe aktualisiert percentText bzw. amountText.
 - Bei Focus-Ende (onFocusChanged) wird onChange(participant.withAmount(...)) oder prozentualer Anteil angepasst.
2. LaunchedEffect synchronisiert externe Änderungen (z. B. bei automatischer Neuberechnung) in die Textfelder, solange kein Fokus besteht.
3. Eingabe-Clamping: Euro-Wert wird auf ≥ 0 begrenzt.

5. SplitMode

Paket: com.example.billsplitter.ui

Typ: enum class

Werte:

- EQUAL = Gleichmäßiger Split
- INDIVIDUAL = Individueller Split

6. SplitUtils

Paket: com.example.billsplitter.util

Typ: Kotlin-Funktion (Utility)

Methoden:

fun redistributeShares(participants: SnapshotStateList<Participant>)

Verteilt die verbleibenden Prozentpunkte (100 % minus aller fixierten Anteile) gleichmäßig auf die unfixed Teilnehmer.

Parameter:

- participants: SnapshotStateList<Participant> – Liste aller Teilnehmer (modifiable)
- Algorithmus:
- totalFixed = sum(participant.percentShare for participant.isFixed)

- `remaining = max(0, 100 - totalFixed)`
- `unfixedIdx = indices where participant.isFixed == false`
- `base = remaining / unfixedIdx.size, rest = remaining % unfixedIdx.size`
- Für jeden Index `i` in `unfixedIdx`:
 - o `share = base + (if rest>0 then 1 else 0); rest--`
 - o `participants[i] = participants[i].copy(percentShare = share)`

7. MainActivity

Paket: `com.example.billsplitter`

Typ: `ComponentActivity`

Attribute:

- Keine eigenen Felder, verwendet nur das Compose-UI

Methoden:

- override fun `onCreate(savedInstanceState: Bundle?)`
Initialisiert die Activity und ruft `setContent { BillSplitApp() }` auf.
- `@Composable fun BillSplitAppPreview()`
Preview-Funktion für Android Studio mit `@Preview-Annotation`.

7. Ordnerstruktur

